

# Soluzioni HPC per il Problema di Laplace

Confronto di 5 approcci: Serial, OpenMP, OpenMP Target, CUDA, MPI



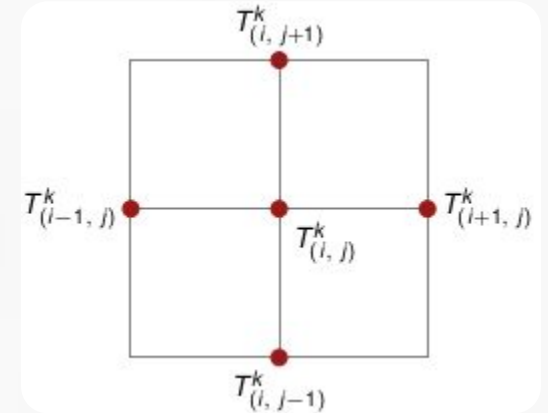
# Introduzione: Problema di Laplace

## Condizioni

- Lattice GLX=512 x GLY=512 con bordi HX,HY
- definiamo una piastra di calore (TH)
- definiamo la temperatura di background (TB)

## Formula

La temperatura di un certo punto della griglia al passo K+1 diventa la media della temperatura dei suoi vicini al passo K



# Hardware utilizzato



Intel(R) Xeon(R) CPU E5-2630L v3 @ 1.80GHz

- 8 Core, 16 Threads
- 60 GB/s max bandwidth
- 230 GFLOPs peak performance

[Fonte-1](#)

[Fonte-2](#)



NVIDIA TESLA V100 GPU

- 5120 cuda cores
- 900 GB/s max bandwidth
- 7000 GFLOPs peak performance

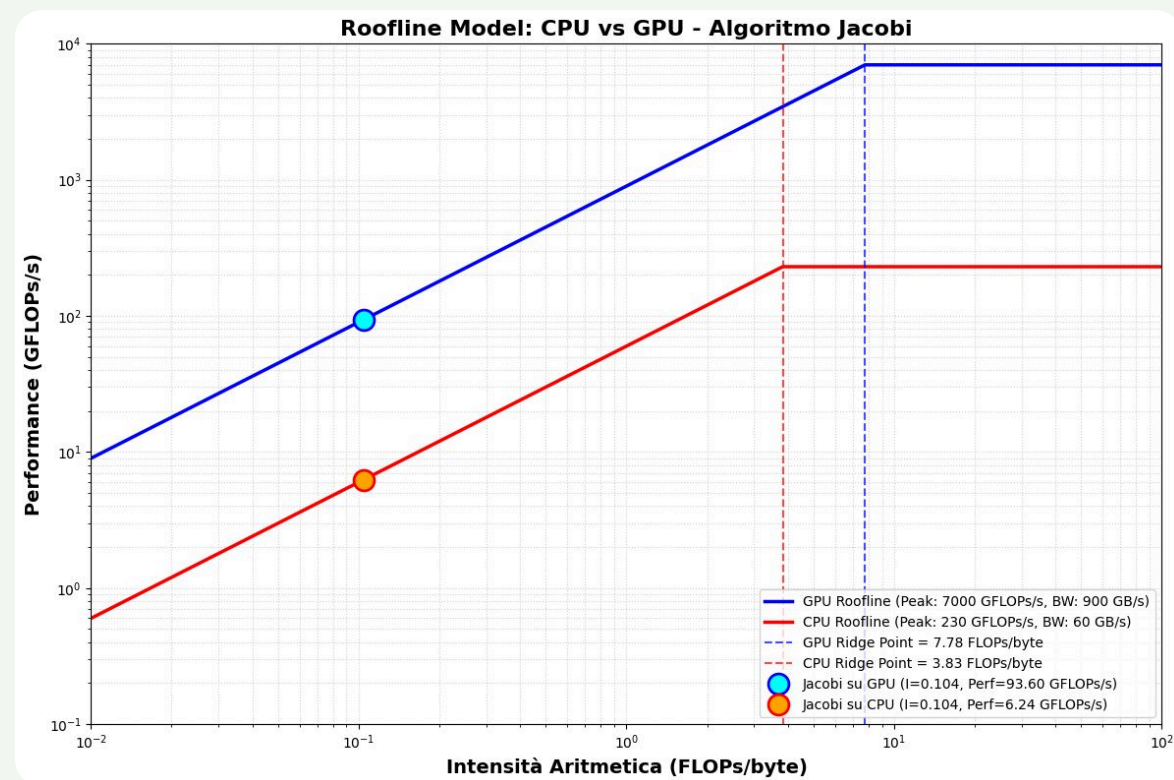
[Fonte-3](#)

# Roofline Model

## Descrizione

- Machine Balance CPU =  $P_{\text{peak}}/B_{\text{peak}} = 230/60 = 3.83$
- Machine Balance GPU =  $P_{\text{peak}}/B_{\text{peak}} = 7000/900 = 7.78$
- Costo computazionale = 4 somme, 1 prodotto in *double precision* =  $5N$  (FLOP)
- Dobbiamo leggere  $5N$  elementi e scrivere  $N$  elementi =  $5N + N$
- Arithmetic Intensity =  $5N / 8 \text{ bytes} * (5N + N) = 0.104$  (FLOP/byte)
- $I < MB$  quindi il nostro algoritmo è **memory-bound**

## Plot



# Versione 1 — Codice Seriale

```
void jacobi_update(double *grid, double *grid_new) {
    for (int i = HY; i < GY - HY; i++) { // - HY così non tocco mai i ghost borders
        for (int j = HX; j < GX - HX - K; j++) { // non aggiorna le colonne della piastra
            int index = i * GX + j; // corrisponde alle coordinate [i,j] nella matrix

            grid_new[index] = (
                grid[(i - 1) * GX + j] + // sopra
                grid[(i + 1) * GX + j] + // sotto
                grid[i * GX + (j - 1)] + // sinistra
                grid[i * GX + (j + 1)] + // destra
                grid[index] // centro
            )/5;
        }
    }
}
```

## Idea

- Loop iterativo sugli elementi della griglia aggiornando la temperatura per ogni punto:
  - esclusi gli halo
  - esclusi i punti interni alla piastra di calore

❏ Limitato per grandi N – tempo e memoria

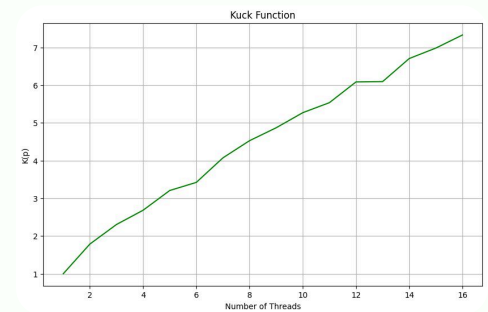
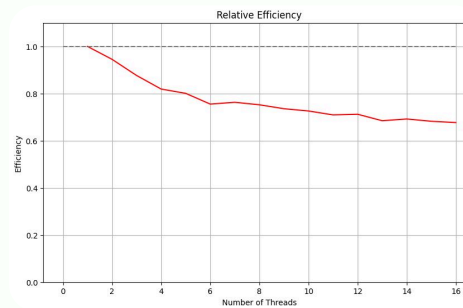
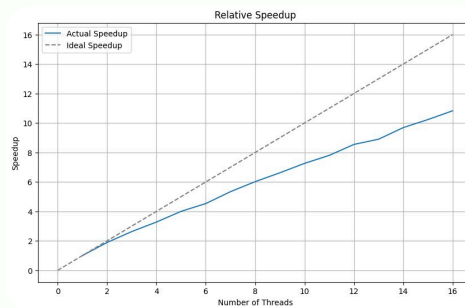
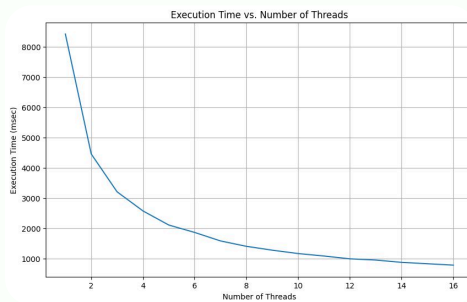
# Versione 2 — OpenMP (CPU)

```
void jacobi_update(double *grid, double *grid_new) {  
    #pragma omp parallel for collapse(2) num_threads(16) // collassa il numero di loop interni  
    for (int i = HY; i < GY - HY; i++) {  
        for (int j = HX; j < GX - HX - K; j++) {  
            int index = i * GX + j;  
            grid_new[index] = (  
                grid[(i - 1) * GX + j] +  
                grid[(i + 1) * GX + j] +  
                grid[i * GX + (j - 1)] +  
                grid[i * GX + (j + 1)] +  
                grid[index]  
            )/5;  
        }  
    }  
}
```

## Strategia

- Slice del lattice lungo l'asse Y
- ogni slice viene processata da un thread
- i bordi delle slice sono visibili perchè i threads hanno memoria condivisa

# Versione 2 — OpenMP (CPU)



# Versione 3 — OpenMP Target (GPU)

## Caratteristiche

- Direttive **#pragma omp** per allocare dati sulla gpu
- Gestione implicita dati host ↔ device
- loop distribuito tra i vari teams
- risultati su gpu v100, skyvolta:

```
void jacobi_update(double *grid, double *grid_new) {
    #pragma omp target teams distribute parallel for collapse(2)
    for (int i = HY; i < GY - HY; i++) { // - HY così non tocco mai i ghost borders
        for (int j = HX; j < GX - HX - K; j++) { // non aggiorno la piastra nelle K-colonne
            int index = i * GX + j; // corrisponde alle coordinate [i,j] nella matrix

            grid_new[index] = (
                grid[(i - 1) * GX + j] + // sopra
                grid[(i + 1) * GX + j] + // sotto
                grid[i * GX + (j - 1)] + // sinistra
                grid[i * GX + (j + 1)] + // destra
                grid[index] // centro
            )/5;
        }
    }
}
```

| allocazione in vram | funzione parallelizzata |
|---------------------|-------------------------|
| 570.35 msec         | 160.65 msec             |
| 22.98 GFLOPs        | 81.59 GFLOPs            |

❏ circa il 71.8% del tempo è "sprecato" per passare i dati alla GPU



# Versione 4 — CUDA (GPU)

```
// Controllo dei limiti per evitare di aggiornare le ghost borders o la piastra nelle K-colonne
global void jacobi_update_cuda(double *grid, double *grid_new) {
    int i = blockIdx.x * blockDim.x + threadIdx.x; // Utilizzando le variabili threadIdx.x e blockIdx.x,
    int j = blockIdx.y * blockDim.y + threadIdx.y;

    // Check bounds
    if (i >= HY && i < GY - HY && j >= HX && j < GX - HX - K) {

        int idx = i * GX + j;
        grid_new[idx] = (
            grid[(i - 1) * GX + j] + // sopra
            grid[(i + 1) * GX + j] + // sotto
            grid[i * GX + (j - 1)] + // sinistra
            grid[i * GX + (j + 1)] + // destra
            grid[idx] // centro
        ) / 5.0;
    }
}
```

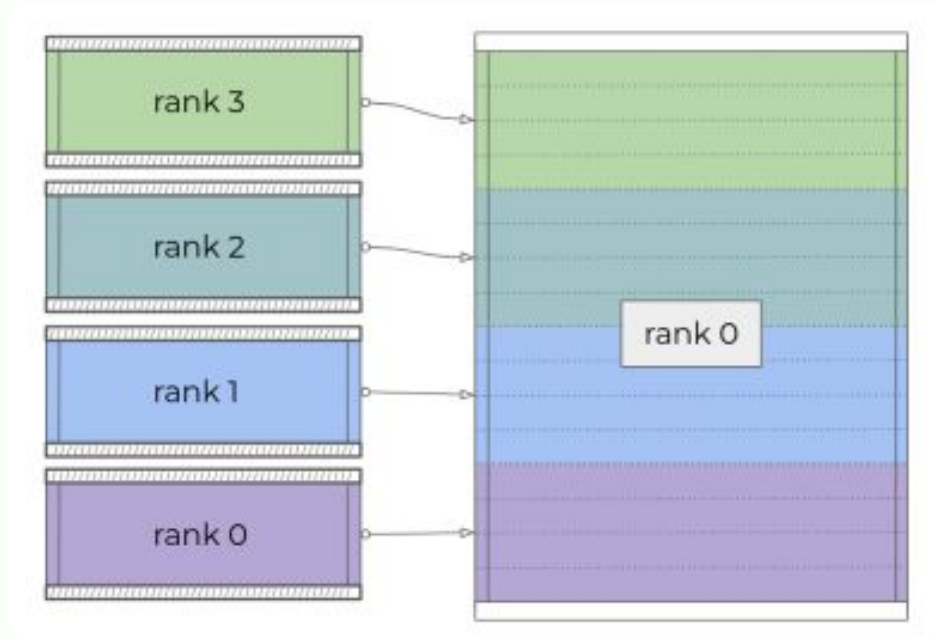
## Caratteristiche

- direttive cudaMalloc e cudaMemcpy per spostare i dati sulla gpu
- Configurazione griglia 32×32
- Configurazione blocchi 16×16
- risultati su GPU v100, skyvolta:

| kernel + cudaMemcpy | kernel       |
|---------------------|--------------|
| 5523.50 msec        | 175.71 msec  |
| 2.37 GFLOPs         | 74.60 GFLOPs |

❏ Circa il 96.8% del tempo è "sprecato" per passare i dati alla GPU

# Versione 5 — MPI (Multi-nodo)



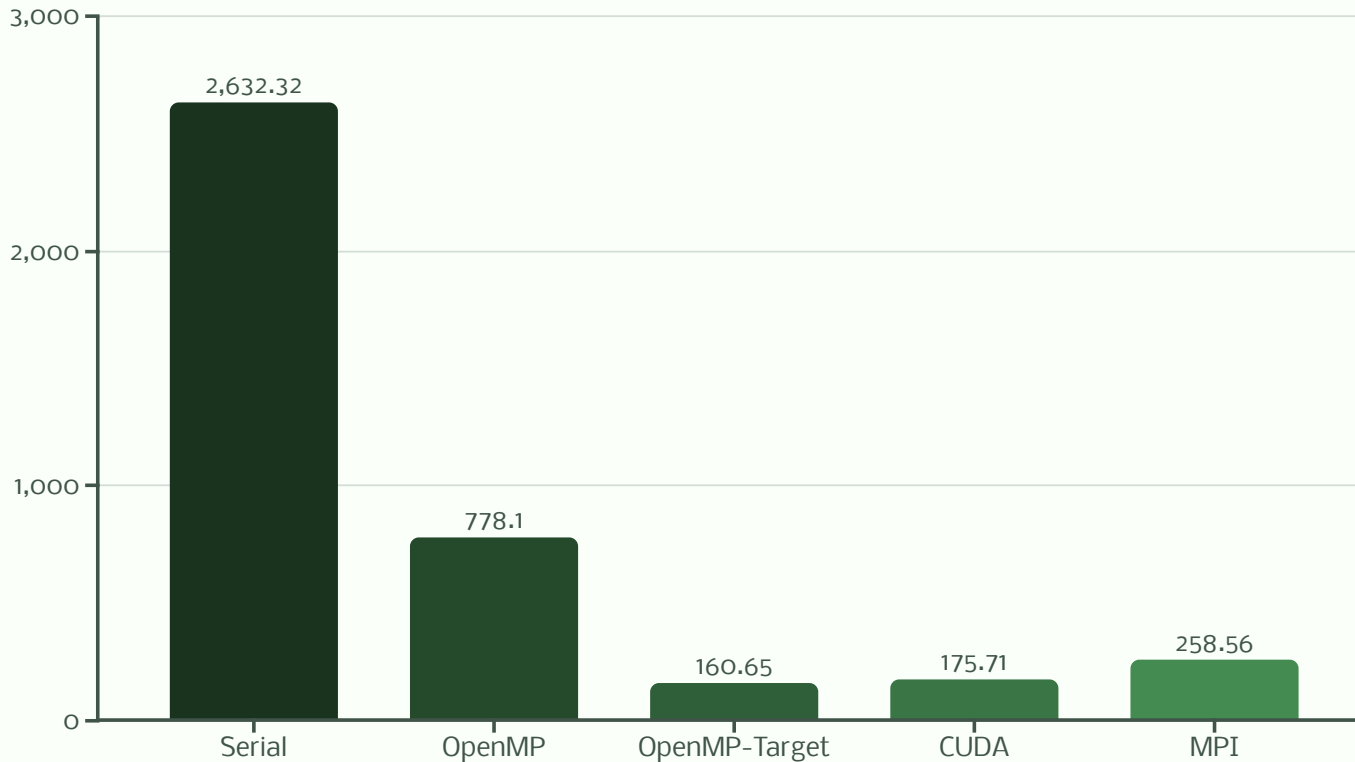
## Strategia

- Ogni processo lavora su una slice del lattice iniziale
- ogni slice contiene un top-halo e un bottom-halo usati per copiare la prima o l'ultima riga (fisica) del rank precedente/successivo
- I risultati degli aggiornamenti vengono aggregati (Gather)

# Risultati MPI

| Processi | Time (ms) | Operations (GFLOPs) |
|----------|-----------|---------------------|
| 2        | 1275.87   | 10.27               |
| 4        | 718.65    | 18.24               |
| 8        | 417.90    | 31.36               |
| 16       | 258.56    | 50.69               |

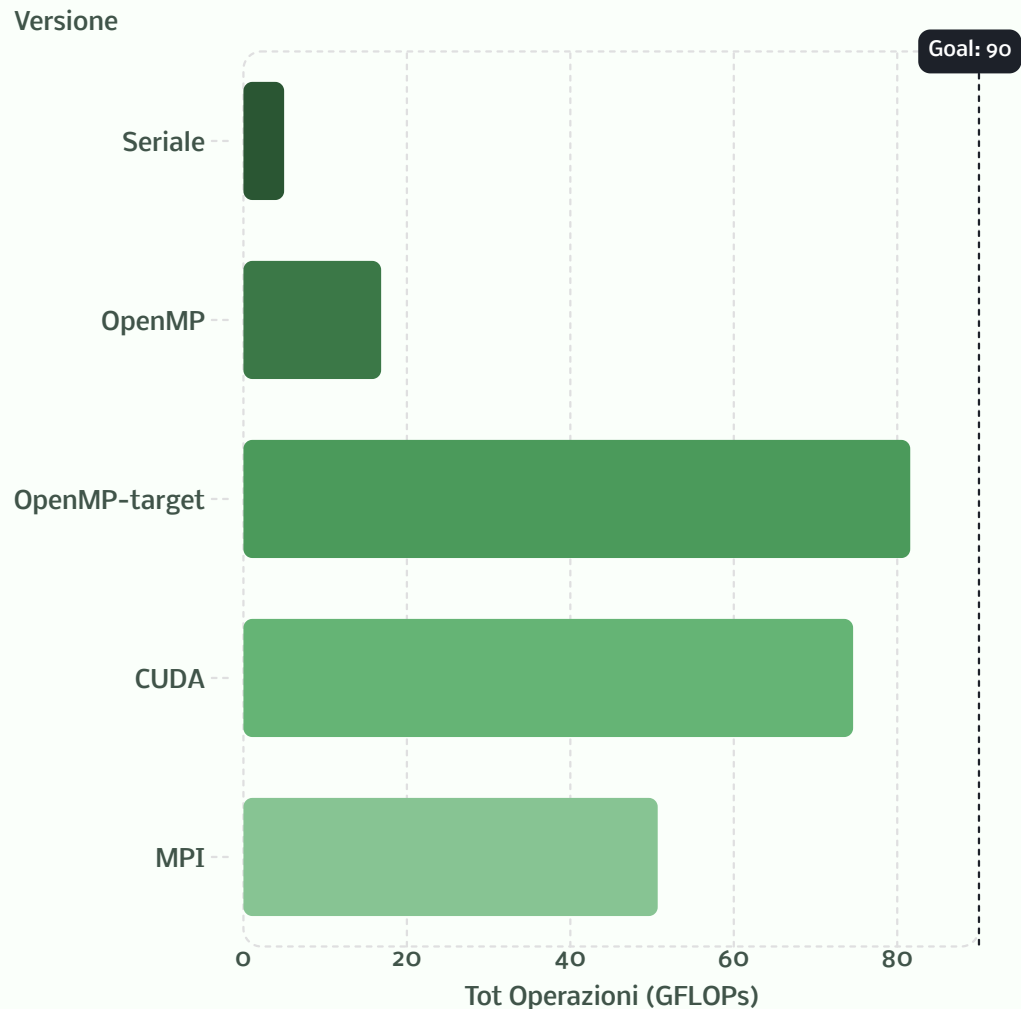
# Performance su loop di aggiornamento



Note:

- MPI: il miglior tempo considerato è su 16 processi
- OpenMP: il miglior tempo considerato è su 16 threads

# Performance su loop di aggiornamento



## Note:

- La versione OpenMP-target si avvicina di più alla performance massima teorica (87%).
- La versione MPI gira con performance molto più elevate rispetto al limite teorico questo può essere dovuto al fatto che ogni rank è interamente allocato in cache e non in RAM

# Cache Miss - MPI

Se aumentiamo le dimensioni del lattice a  $2048 \times 2048$  dovremmo occupare circa 32MB che non stanno in cache L3 costringendo la CPU ad effettuare accessi in RAM

| Processor | Time(ms)  | Operations(GFLOPs) |
|-----------|-----------|--------------------|
| 2         | 35,790.25 | 5.86               |
| 4         | 28,604.46 | 7.33               |
| 8         | 27,288.41 | 7.69               |
| 16        | 13,619.96 | 15.40              |

- con 16 processi le slices su cui lavorano ogni processo diventa  $\Rightarrow NX : 2048 + 2$  (di ghost cells) = 2050 ;  $NY = 2048 / 16 + 2 = 130$ ;  $NX \times NY = 2050 \times 130 \times 8 \text{ bytes} = \text{circa } 2\text{MB}$

# Cache Miss - MPI

Se aumentiamo le dimensioni del lattice a  $8192 \times 8192$  dovremmo occupare circa 512MB che non stanno in cache L3 costringendo la CPU ad effettuare accessi in RAM

| Processor | Time(ms)   | Operations(GFLOPs) |
|-----------|------------|--------------------|
| 8         | 435,719.32 | 7.70               |
| 16        | 218,810.17 | 15.33              |

# Cache Miss - GPU

Se aumentiamo le dimensioni del lattice a  $2048 \times 2048$  dovremmo occupare circa 32MB che non stanno in cache costringendo la GPU ad effettuare accessi in memoria globale

## OpenMP-target

- dT: 1022.55 msec
- GFLOPs: 205.09

## CUDA

- dT: 1315.77 msec
- GFLOPs: 159.39



# Project Work : Jacobi strided

Una variante dell'algoritmo di jacobi per risolvere il problema di Laplace è l'implementazione di un kernel cuda con funzionalità strided ovvero andiamo si estende la possibilità di utilizzare:

- Dimensione della griglia variabile
- Dimensione dei blocchi variabile

📄 Nell'approccio precedente con un lattice  $512 \times 512$  era possibile testare solo 2 configurazioni griglia  $32 \times 32$  con dimensioni dei blocchi  $16 \times 16$  oppure la combinazione inversa

# Kernel strided

```
global __void jacobi_update_cuda(double *grid, double *grid_new) {
    int tidx = threadIdx.x;
    int tidy = threadIdx.y;

    int bidx = blockIdx.x;
    int bidy = blockIdx.y;

    int bdimx = blockDim.x;
    int bdimy = blockDim.y;

    int gdimx = gridDim.x;
    int gdimy = gridDim.y;

    // Coordinate globali iniziali
    int i_start = bidx * bdimx + tidx; // r
    int j_start = bidy * bdimy + tidy; // c

    // Calcolo stride in entrambe le direzioni
    int stride_i = gdimx * bdimx;
    int stride_j = gdimy * bdimy;

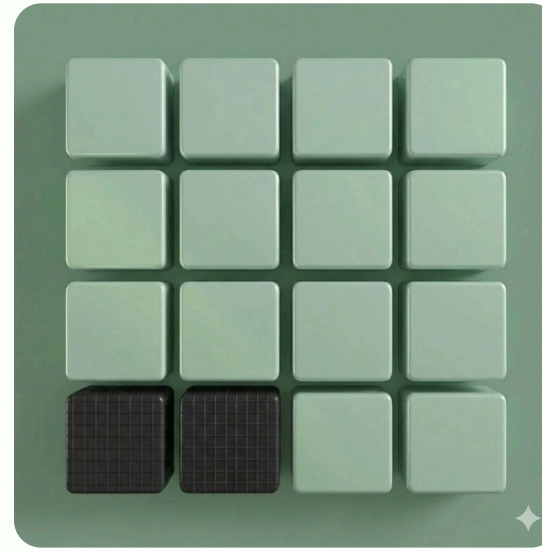
    for (int i = i_start; i < GY; i += stride_i) {
        for (int j = j_start; j < GX; j += stride_j) {

            // Escludiamo ghost borders e la piastra fissa a destra
            if (i >= HY && i < GY - HY && j >= HX && j < GX - HX - K) {
                int idx = i * GX + j;

                grid_new[idx] = (
                    grid[(i - 1) * GX + j] + // sopra
                    grid[(i + 1) * GX + j] + // sotto
                    grid[i * GX + (j - 1)] + // sinistra
                    grid[i * GX + (j + 1)] + // destra
                    grid[idx] // centro
                ) / 5.0;
            }
        }
    }
}
```

Caratteristiche:

- I blocchi cuda possono essere quadrati o rettangolari
- La griglia cuda può coprire solo una porzione del lattice
- Più combinazioni DimGrid, DimBlock testabili



# Hardware utilizzato



AMD Instinct MI210

- 6,656 stream processor
- 1.6 TB/s max bandwidth
- 22.6 TFLOPs peak performance
- L1 Cache 16 KB,
- L2 Cache 8 MB

[Fonte-2](#)



NVIDIA TESLA V100 GPU

- 5120 cuda cores
- 0.9 TB/s max bandwidth
- 7 TFLOPs peak performance
- L1 Cache 128 KB (per SM)
- L2 Cache 6 MB

[Fonte-3](#)

# Scrittura degli scripts

## AMD

Compilatore **hipcc**

hipMalloc → alloca uno spazio di memoria sul device

hipMemcpy → copia i dati da cpu a gpu

## NVIDIA

Compilatore **nvcc**

cudaMalloc → alloca uno spazio di memoria sul device

cudaMemcpy → copia i dati da cpu a gpu

# Optuna

Libreria Python per l'ottimizzazione di un algoritmo tramite ricerca dinamica degli iperparametri

1. Iperparametri → DimGrid (da 32 a 2048), DimBlock (da 32 a 1024)
2. Funzione obiettivo → Tempo di esecuzione del kernel (dT)
3. numero di combinazioni testate → 800 combinazioni su lattice  $1024^2$
4. vincolo optuna → *if nthreads\_x \* nthreads\_y > 1024: raise optuna.TrialPruned()*



# Risultati degli esperimenti: **AMD**

## Optuna results: Best trial

- Lattice  $\rightarrow 1024 \times 1024$
- Completed trials: 652/800
- DimBlock  $\rightarrow$  nthreads\_x: 64, nthreads\_y: 1
- DimGrid  $\rightarrow$  num\_blocks\_x: 64, num\_blocks\_y: 256
- Time : **353.33** msec

## Cache Miss experiments

| lattice   | Time (ms)           |
|-----------|---------------------|
| $2048^2$  | $7.348 \times 10^3$ |
| $4096^2$  | $4.984 \times 10^4$ |
| $8192^2$  | $2.539 \times 10^5$ |
| $16384^2$ | $1.324 \times 10^6$ |

# Risultati degli esperimenti: Nvidia

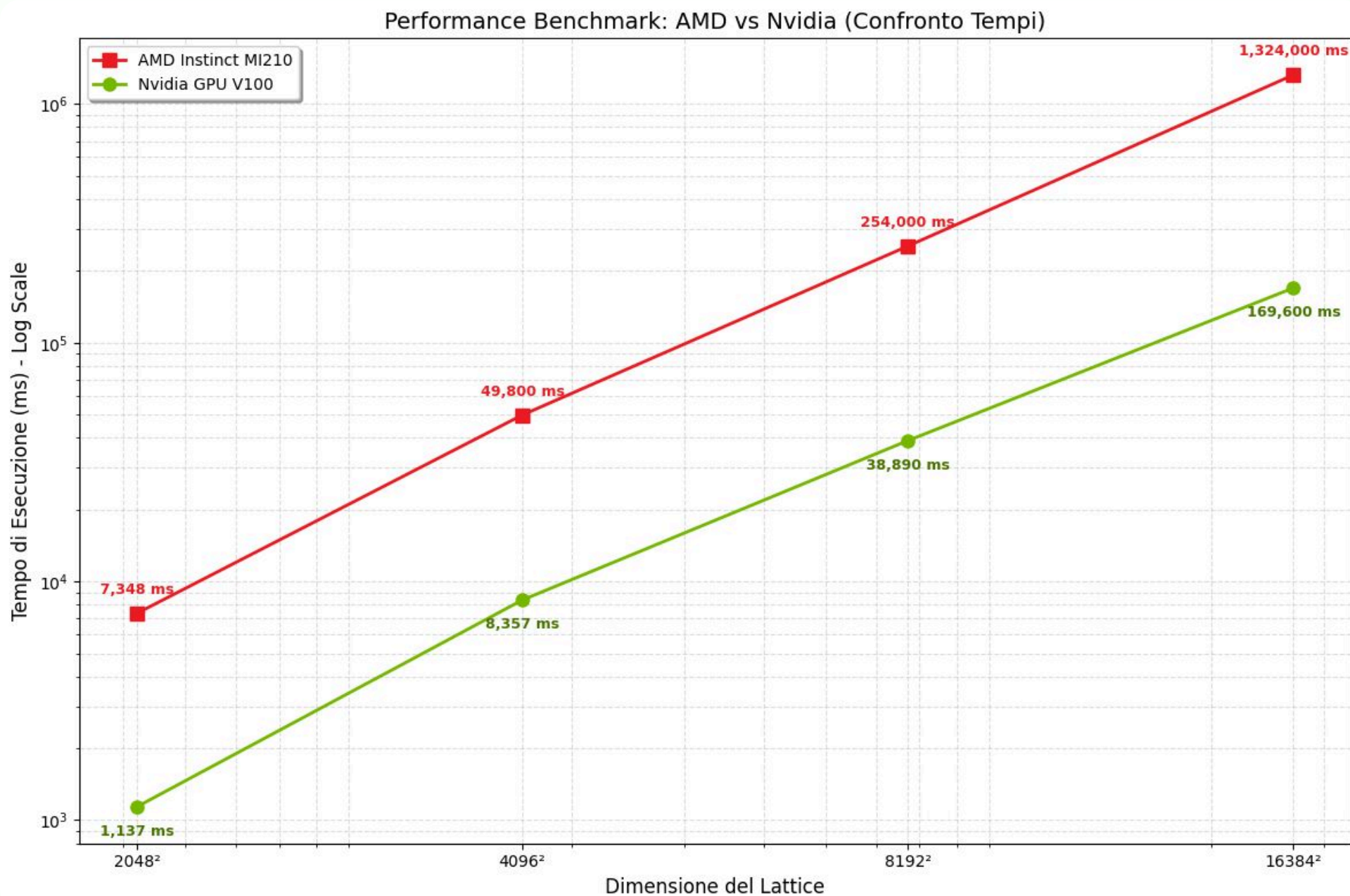
## Optuna results: Best trial

- lattice  $\rightarrow 1024 \times 1024$
- DimBlock  $\rightarrow$  nthreads\_x: 2 nthreads\_y: 32
- DimGrid  $\rightarrow$  num\_blocks\_x: 2048 num\_blocks\_y: 32
- Time: **349.29** ms

## Cache Miss experiments

| lattice   | Time (ms)           |
|-----------|---------------------|
| $2048^2$  | $1.137 \times 10^3$ |
| $4096^2$  | $8.357 \times 10^3$ |
| $8192^2$  | $3.889 \times 10^4$ |
| $16384^2$ | $1.696 \times 10^5$ |

# Grafico di confronto dei risultati



- Il tempo di esecuzione del kernel cuda è nettamente inferiore al tempo registrato su gpu amd, questo risultato potrebbe essere dovuto al fatto che la **cache L1** di nvidia V100 è **8 volte** più grande rispetto a quella amd, riducendo i tempi di latenza negli accessi alla memoria.